Exercise 16: Balance Control Random Testing

- You will perform random testing on your **balance_cntrl.sv** unit
- We provide you with a file that contains random stimulus and another with the expected response
  - Download **balance_cntrl_stim.hex** and **balance_cntrl_resp.hex** from the Canvas page.
- You are to create a testbench **balance_cntrl_chk_tb.sv** that applies these inputs to your DUT, and check your outputs against the expected response

balance_cntrl_stim.hex

49-bit vector should be assigned as follows:

Stimulus Bit Range	Signal Assignment
stim[48]	rst_n
stim[47]	vld
stim[46:31]	ptch
stim[30:15]	ptch_rt
stim[14]	pwr_up
stim[13]	rider_off
stim[12:1]	steer_pot
stim[0]	en_steer

balance_cntrl_resp.hex

25-bit vector should be assigned as follows:

Response Bit Range	Signal Assignment
resp[24:13]	lft_spd
resp[12:1]	rght_spd
resp[0]	too_fast

Exercise 16: Balance Control Random Testing

- There are 1500 vectors of stimulus and response. Read each file into a separate "memory" using **$readmemh**.
  - Declare a memory large enough to hold 1500 49-bit vectors, and another large enough to hold 1500 25-bit vectors.
  - Inside your initial block in the testbench, do a $readmemh of the provided .hex files
- Use a force statement to make **ss_tmr full** for this test
  - In your initial block, have: force <DUT name>.ss_tmr = 8'hFF;
- Make sure your **fast_sim = 1** for this test
- Loop through the 1500 vectors and apply the stimulus vectors to the inputs as specified. Do all 1500 vectors match?
  - Start clk at 0 and toggle it every #5 time units
  - Use a **for** loop to go over 1500 entries, assigning 1 entry of the stim memory as the vectors that drive the DUT inputs
  - Wait for @(posedge clk), then wait for #1 more time unit. Now, check if the outputs match the expected output response vector
  - Debug your DUT if you did not pass all of them. You may want to go back to PID_fastsim_tb.sv from before
- **Submit**: balance_cntrl.sv, balance_cntrl_chk_tb.sv